

Leant2: Answers to the Questions for Experts

Search, carrier invention, dependent elimination,
recursive synthesis, and trustworthy certification

An investigation of all 31 questions in

`docs/proposals/11-further-improvements`

Prepared for Vladimir Reshetnikov

21 September 2026

Repository snapshot

`https://github.com/VladimirReshetnikov/Leant2`

Commit `784664ff34e819422510a4d470ab07fe48bb736b`

Repository toolchain: Lean 4.34.0; core-only default.

Evidence and validation. This report combines primary-source research, static inspection of the pinned proposal and selected implementation files, mathematical arguments, and executed finite Python checks. It is not a report of a completed Leant2 implementation, a Lean-checked formalization of the arguments, or a rerun of the repository's synthesis benchmarks. Proposed algorithms and unmeasured performance expectations are identified as such.

Abstract

The further-improvements proposal identifies the right bottlenecks, but several of its research questions have more definite answers than the document suggests. Smyth already supports higher-order function examples in its implementation; Hoogle+ already incorporates type-class dictionaries; auxiliary-summary invention has close precedents in AutoLifter; and Lean exposes reusable motive construction. These findings substantially narrow, but do not eliminate, the adaptation work.

The main recommendation is to develop Leant2 around three explicit objects: a joint constraint state, a typed recursion or carrier sketch, and a certificate of each decisive conclusion. This separates candidate correctness from safe pruning, bounded coverage, ranking optimality, and reproducibility. We give constructions for minimal finite sample-consistent carriers, prove a residual state lower bound and a parametric shape–position representation theorem, explain why reversing a list is already a fold into lists, and formulate completion-stable residual evaluation. We also give concrete Lean integration points, a dependency-aware proof schedule, a type-class-aware retrieval design, and a regression and evaluation plan. Every expert question receives an individual answer. General guarantees are explicitly restricted to their stated fragments; empirical questions are not answered with invented measurements.

Contents

1	Scope, evidence, and the guarantees that must be separated	3
1.1	What was investigated	3
1.2	A concise answer map	3
1.3	Five guarantees, not one overloaded notion of soundness	4
1.4	Corrections that change the plan	4
2	R1: Search with coupled goals and bounded resources	5
2.1	R1.Q1. Can independent work be shared without copying entire branches?	5
2.2	R1.Q2. What admissible heuristic survives shared unification?	6
2.3	R1.Q3. Can just-in-time learning coexist with principled ranking?	6
2.4	R1.Q4. Can a wall-clock deadline give machine-independent results?	7
3	R2: Carrier invention, auxiliary state, and generalization	7
3.1	R2.Q1. Can finite examples determine a minimal carrier type?	7
3.2	R2.Q2. What is the correct polymorphic alphabet?	9
3.3	R2.Q3. Can fusion be run backward to discover auxiliary functions?	10
3.4	R2.Q4. Is there a complete practical motive enumeration?	11
3.5	R2.Q5. Do theorem-prover generalization heuristics transfer?	12
4	R3: Evaluation, hole closures, and safe refutation	12
4.1	R3.Q1. Does live bidirectional evaluation already handle higher order?	12
4.2	R3.Q2. What can be reused for assignment-driven memoization?	13
4.3	R3.Q3. What is the cheapest trustworthy fast evaluator?	13
4.4	R3.Q4. Can top-down angelic recursion terminate completely?	14
5	R4: Indexed families, motives, and transports	15
5.1	R4.Q1. Which motive searches are finite and complete?	15
5.2	R4.Q2. How can transports be normalized without losing meaning?	16
5.3	R4.Q3. What do Canonical and sauto actually do about unfolding?	16
5.4	R4.Q4. Which Lean APIs can be reused now?	17

6	R5: Recursive-call capabilities and realization	18
6.1	R5.Q1. Does a search-time interpreter agree definitionally with Lean?	18
6.2	R5.Q2. Does angelic execution compose with top-down synthesis?	19
6.3	R5.Q3. Is there a canonical small recursion basis with known coverage?	19
7	R6: Universal contracts and program–proof interleaving	20
7.1	R6.Q1. How should value holes and proof holes be interleaved?	20
7.2	R6.Q2. Can the easy induction class be recognized?	20
7.3	R6.Q3. Which Lean induction automation is usable?	21
7.4	R6.Q4. Is there a principled proof-debt scheduler?	22
8	R7: Retrieval over dependent and class-polymorphic libraries	22
8.1	R7.Q1. Which type abstraction is useful and conservative?	22
8.2	R7.Q2. Has type-guided abstraction refinement handled type classes?	23
8.3	R7.Q3. Can Lean’s selector take data goals and examples?	24
9	R8: Ranking, equivalence, and stopping certificates	25
9.1	R8.Q1. Is there a defensible objective for the first answer?	25
9.2	R8.Q2. Which candidate equivalence is cheap and meaningful?	25
10	R9: Learned proposals without a new trust dependency	26
10.1	R9.Q1. What accuracy do current models achieve on carriers and helpers?	26
10.2	R9.Q2. How can the contribution be deterministic and auditable?	27
11	A unified implementation and research plan	27
11.1	One joint state, several certified services	27
11.2	Suggested module boundaries	28
11.3	Implementation order and exit criteria	29
11.4	Adversarial tests that should accompany the happy-path suite	30
11.5	What the evaluation should report	31
11.6	Overall conclusion	31
A	Executed checks and reproducibility	31
A.1	What was actually executed	31
A.2	Rebuilding the package	32
A.3	Source and uncertainty boundaries	32

1 Scope, evidence, and the guarantees that must be separated

1.1 What was investigated

The target is the repository at commit `784664ff34e819422510a4d470ab07fe48bb736b`, not a hypothetical rewrite of Leant2. The expert questions occupy research items R1–R9 in the proposal; R8 and R9 share `sections/12-ranking.tex`. The later expert-summary section prioritizes these questions but does not add a separate question set. The pinned README identifies Lean 4.34.0 and a core-only implementation [1, 2].

All nine question groups were read, together with the adaptation discussion, reference list, and expert-priority summary. Selected implementation inspection covered the transactional state, acceptance gate, and the provider and residual evaluation portions of the search core. This is a targeted architecture review, not an exhaustive audit of every search rule or frontend caller [3, 4, 5]. Literature claims below are tied to papers, authors’ artifacts, or official documentation. Several particularly relevant works postdate the older synthesis literature, including the 2024 work on polymorphic example realizability and the 2026 Canonical-min paper [17, 10].

The report distinguishes three kinds of statements. *Established results* are attributed to their sources. *Arguments in this report* are presented with assumptions and proofs; no claim of research priority or machine checking is implied. *Proposals* specify an implementation or experiment that has not been run in Leant2. This distinction matters most when discussing unknown model accuracy, expected speedups, and library integration costs.

1.2 A concise answer map

Table 1 is both a coverage checklist and the executive recommendation. Question numbers below preserve the order within each original expert section. The headings paraphrase the questions rather than reproducing the proposal.

Table 1: Coverage of the 31 expert questions.

Group	Count	Principal answer and destination
R1	4	Joint states; admissible relaxations; fixed ranking objectives; reproducible logical prefixes (Section 2).
R2	5	Finite minimality, certified parametricity, bounded typed grammars, and counterexample-driven lifting (Section 3).
R3	4	Existing higher-order Smyth; typed closures, versioned dependencies, and checked refutations for Lean (Section 4).
R4	4	Finite motive templates, explicit transports, controlled unfolding, and Lean induction APIs (Section 5).
R5	3	Scoped recursive capabilities; interpreter–realization agreement; a measured rather than assumed scheme basis (Section 6).
R6	4	Dependency-sensitive proof jobs; sufficient induction templates; bounded, fair proof scheduling (Section 7).
R7	3	Dependent hyperedges and dictionary obligations; a data-oriented adapter around library selection (Section 8).
R8	2	An explicit ranking objective; exact equality distinguished from finite probe agreement (Section 9).
R9	2	Direct model accuracy remains unmeasured; typed proposals and receipts make contributions auditable (Section 10).

1.3 Five guarantees, not one overloaded notion of soundness

Let a request consist of a type T , an optional contract $C : T \rightarrow \text{Prop}$, an allowed environment E , and a trust profile A . A completed accepted answer must contain a term $p : T$ and, when requested, a proof $\pi : C(p)$ whose permitted axiom dependencies satisfy A . Call this *acceptance correctness*. The kernel is the final authority for these judgments, relative to the chosen axioms. It says nothing by itself about candidates that were never submitted.

Pruning safety is a different obligation: deleting a partial program must not delete an allowed completion satisfying the request. An erroneous refuter may preserve the correctness of every returned answer while making search systematically miss solutions. *Bounded coverage* additionally requires that every candidate in the advertised finite grammar is generated or safely excluded. A timeout is not an exclusion proof.

Ranking optimality means optimality for a stated objective over a stated candidate set. It does not follow from kernel acceptance or from finding the first candidate under an unrelated search heuristic. Finally, *reproducibility* concerns stable results or replay under specified resource and environment assumptions. None of these five properties should be reported using an undifferentiated “sound” flag.

Definition 1.1 (Completion set). For a well-scoped partial program p in joint state S , write $C(S)$ for the substitutions of its unresolved objects that satisfy all typing, universe, local-context, recursion, and declared grammar constraints. A safe contract refutation establishes

$$\forall \theta \in C(S), \quad \neg C(p\theta). \quad (1)$$

Membership in a finite resource-bounded search class is an additional restriction, not an implicit part of Lean’s type theory.

A final gate should therefore be indexed by the *original request*. The inspected gate already performs synchronous checking, universe generalization, and transitive axiom auditing. Its interface receives a program and type and an optional proof and proof type, rather than the original contract itself [4]. A stronger interface should derive the expected proposition $C(p)$ from the stored request and verify the supplied proof against that proposition. This is an interface-hardening recommendation, not a demonstrated bug in a frontend caller that has not been audited.

1.4 Corrections that change the plan

Several corrections recur throughout the report. Counting open goals is not generally an admissible cost heuristic. Reversal does not require a new carrier for semantic expressibility: a right fold with list-valued state and a snoc step already computes it. Partial observation tables cannot count missing entries as differences. Testing closed programs cannot prove a normalizer safe on all open completions. Proof irrelevance does not authorize dropping arbitrary transports. A finite sample signature is not global program equality.

There are also positive literature corrections. Smyth’s implementation supports function examples beyond the first-order presentation of its core; its synthesis method does not require trace-complete examples. Hoogse+ has an explicit dictionary treatment of type classes. Existing synthesis work studies auxiliary summaries and unrealizability witnesses, so carrier invention should not be framed as an entirely unoccupied research area [12, 14, 15, 16]. The remaining challenge is making these ideas cooperate over exact Lean expressions within a small interactive budget.

2 R1: Search with coupled goals and bounded resources

Proposal source: sections/05-search.tex in [2].

2.1 R1.Q1. Can independent work be shared without copying entire branches?

Answer. Yes, as a representation and constraint-decomposition strategy; no, if the question demands that correlations disappear without being represented. The appropriate object is an AND/OR graph of *joint constraint states*, with persistent storage and conditional subproblem summaries. Independence is a property of the full constraint state, not merely disjoint metavariable names in the printed goal types.

Aesop already discusses transitive metavariable-coupled goal clusters and the need to isolate assignments between alternatives. Consequently, discovering such clusters is not itself a new departure from Aesop [8]. A directly relevant newer comparison is Canonical-min: it presents inhabitation and unification using suspended constraints and allows refinement of later metavariables before the metavariables on which their types depend. Its completeness claim concerns its presented calculus, not arbitrary Lean 4.34 features, user tactics, or deadline-bounded synthesis [10].

For Leant2, build a hypergraph whose vertices are unresolved expression and universe metavariables. Add an edge for each constraint that can read or assign several vertices. The dependency closure must include the types and values of local declarations, delayed assignments, postponed unification constraints, and unresolved instance arguments. A rule’s provider types and admissible local expressions also belong in its read set. Looking only at `collectMVars` on a target is insufficient.

Separate three levels of reuse. First, immutable expression and context nodes can always be physically shared. Second, alternatives can share a persistent prefix while carrying different assignment maps. Third, a solved subproblem can be reused semantically when its interface assumptions match. For the third, store a contextual term: abstract its free local variables over a telescope, record any remaining interface variables, and reinstantiate and type-check it at the destination. Never move a raw expression containing branch-local `FVarIds` into an unrelated local context.

Proposition 2.1 (Conditional component factorization). *Suppose the constraints of a state, after fixing interface assignments η , partition as $\Phi_1(X_1; \eta), \dots, \Phi_k(X_k; \eta)$ with pairwise disjoint sets X_i . Suppose every allowed continuation of component i can read shared immutable data and η , but cannot read or assign X_j for $j \neq i$. Then the completion set at η is the Cartesian product of the component completion sets. In particular, independently obtained solutions can be combined without a further search over their internal assignments.*

Proof. An assignment to the disjoint union of the X_i is uniquely a tuple of assignments to the individual X_i . By the read/write assumption, satisfying Φ_i depends only on its own tuple entry and η . Thus all constraints hold exactly when every component constraint holds. The same assumption ensures that subsequent refinement does not invalidate this factorization. $\square$

Without fixed η , the result is a union of such products, indexed by compatible interface assignments. A memo entry can therefore be a relation between interface assignments and solutions rather than one globally committed solution. This resembles separator-based constraint decomposition. It does not magically remove the potentially exponential number of interface alternatives; it avoids copying their common representation.

Recommendation for Leant2. Start with whole-state snapshots and dynamic coupling components, then add reusable *closed contextual solutions*. Introduce conditional solution tables only after measuring repeated work. Separate a successful-term cache from a failure cache: “no solution under this grammar and complete bound” may be reusable; “not found before timeout” is not a semantic failure fact.

2.2 R1.Q2. What admissible heuristic survives shared unification?

Answer. The universally safe starting point is $h(S) = 0$. A useful improvement must be the cost of a *relaxation* of the complete state, not a sum of informally counted holes. One assignment may settle several obligations, including indices and instance outputs. Moreover, the proposal gives some closing operations zero cost. An arbitrary unit charge for each still-open non-leaf goal therefore has no general admissibility guarantee.

Fix the objective first. If $J(p)$ is the weighted construction cost of a final program, the cost of failed attempts is not part of J . If the objective is expected search time, estimating that time is valuable, but it is not an admissible lower bound for J . These objectives should not share an unexplained “cost” field.

Let $d(S)$ be the cheapest completion cost under the fixed grammar. An abstract state $\alpha(S)$ is useful for admissible search when every concrete completion maps to an abstract completion with no greater cost. Then

$$h(S) = d^\#(\alpha(S)) \leq d(S). \quad (2)$$

Possible abstractions forget indices, treat some proof and dictionary obligations as free, and allow more providers than the concrete language. Such a model can be weak, but its weakness is in the safe direction. Its transitions must faithfully over-approximate all admitted concrete rules; a type-head graph that silently omits a cast or higher-order introduction cannot justify pruning.

For independently justified lower bounds h_i on individual obligations, $\max_i h_i$ is safe for a joint completion: any joint completion must meet each individual lower bound. The sum is safe only after proving an additive decomposition or partitioning each concrete action’s cost among abstractions so that the total charge never exceeds that action’s real cost. Merely finding disjoint target expressions is not that proof.

Maintain a second, explicitly heuristic priority for estimated branching, likely contract progress, and expected cost of evaluation. It may decide which node to expand, while a separate lower-bound ledger tracks what remains unexplored. Then best-first exploration need not pretend to be optimal A*. Zero-cost rules should either be terminating normalization steps or be guarded by a well-founded progress measure and duplicate detection. Otherwise an infinite zero-cost expansion can prevent every positive-cost candidate from being visited.

2.3 R1.Q3. Can just-in-time learning coexist with principled ranking?

Answer. Yes, but it changes discovery order and can change the returned set under a finite budget. That is not a logical unsoundness; it is a policy effect that needs an explicit user-facing meaning. PROBE supplies a precedent for learning enumeration priorities from partial successes during a query, not a theorem that deadline-limited rankings are unchanged [11].

Use separate fields for a fixed final objective J , a proved lower bound on unseen J , and a learned expansion score L_t . Record the version of L_t in each scheduling epoch. If changing the model changes queue ordering, rekey the queue or apply the new score only at a deterministic epoch boundary; stale priorities must not support a claim of cost-optimal termination. Updating production costs halfway through a search and continuing to call the search “A* under one grammar” is especially misleading.

A fair baseline lane is useful even when learning is excellent. Allocate a fixed positive share of logical work to enumeration in an immutable grammar order. Learned proposals receive the remaining share. With unbounded execution, finite branching, fair admission, and suitable local termination assumptions, every finite baseline derivation is eventually visited. This does not promise that all candidates up to some size are visited in ten seconds.

The original final ranking keys need not change when learning changes. Report “best certified candidates found under this budget,” and report a stronger optimality result only when the lower-bound ledger proves it. This is more predictable than silently redefining “best” to mean “favored by the current within-query model.”

2.4 R1.Q4. Can a wall-clock deadline give machine-independent results?

Answer. Not in general while also returning all improvements discovered before the deadline. The obstruction is elementary and cannot be removed by a restart schedule or startup calibration.

Proposition 2.2 (Deadline obstruction). *Consider a deterministic algorithm with a sequence of logical steps and a candidate first available after step k . Suppose two machines, given the same deadline, can complete respectively fewer than k and at least k steps. A policy that reports that candidate whenever discovered before the deadline cannot return the same candidate set on both machines.*

Proof. The slower execution has not yet produced the candidate and the faster one has. The stated reporting rule therefore includes it only in the latter set. $\square$

Identical results are possible under additional assumptions: a known lower bound on hardware throughput, a fixed amount of logical work that always finishes, withholding results until fixed checkpoints, or replay of a completed receipt. Without such assumptions, even checkpoint policies may reach different checkpoints before a deadline. A useful promise is instead that equal receipts and equal logical budgets give equal outcomes, and that deadlines only truncate a stable logical event stream.

For Leant2, count deterministic work events such as attempted rule instances, constraint propagation steps, and proof-tier quanta. Preserve wall time as a cancellation limit, not as the source of rule ordering. Record the last completed checkpoint and pending work in the receipt. Iteration order, tie breaks, generated names, parallel result admission, and cache effects must be controlled if they can alter that event stream. Calibration can improve responsiveness but is not an equality proof.

Finally, a universal restart result for randomized algorithms does not justify repeatedly running the same deterministic search prefix. A persistent frontier that resumes interrupted work is the direct remedy for that particular waste; restarts should introduce a genuine new policy or random trial, with its own stated assumptions.

3 R2: Carrier invention, auxiliary state, and generalization

Proposal source: sections/06-carriers.tex in [2].

3.1 R2.Q1. Can finite examples determine a minimal carrier type?

Answer. Yes in an explicitly finite model, and yes relative to a completely bounded typed program grammar. Not as an unrestricted decision procedure for arbitrary Lean types and arbitrary step and finishing functions. The first necessary clarification is what “minimal” means.

State cardinality, the syntax size of a carrier type, the code size of its operations, and the runtime of the resulting program are different objectives. A pair of natural numbers can be encoded by a natural number, so cardinality alone cannot justify preferring Nat over $\text{Nat} * \text{Nat}$. The encoding may merely move substantial work into the step and finishing functions. Similarly, a function carrier can be syntactically small while its closures retain an input-sized amount of information. Choose a declared joint objective on $(K, \text{init}, \text{step}, \text{finish})$, not an unexplained ordering of type names.

The exact residual-state problem

For a total function $h : \text{List } A \rightarrow B$, define

$$s \sim_h t \iff \forall u : \text{List } A, h(u ++ s) = h(u ++ t). \quad (3)$$

The continuation is a *prefix* because a right fold builds the result by prepending elements to the already processed suffix. Confusing this orientation with left-to-right streaming changes the equivalence being computed.

Theorem 3.1 (Residual lower bound and quotient realization). *The relation $\sim_h$ is an equivalence relation stable under prepending an element. If*

$$h(s) = o(q(s)), \quad q([]) = q_0, \quad q(a :: s) = \delta(a, q(s)), \quad (4)$$

then $q(s) = q(t)$ implies $s \sim_h t$. Consequently any r pairwise $\sim_h$ -inequivalent lists require at least r distinct reachable carrier values. Conversely, the quotient $\text{List}A/\sim_h$ carries a well-defined right-fold realization of h .

Proof. Reflexivity, symmetry, and transitivity follow pointwise from equality in B . If $s \sim_h t$, then for any u , $h(u ++ (a :: s)) = h((u ++ [a]) ++ s)$ equals $h((u ++ [a]) ++ t)$. This proves stability under prepending. If $q(s) = q(t)$, induction on u gives $q(u ++ s) = q(u ++ t)$, and applying o gives the desired output equality. For the quotient, take initial state $[[[]]$, transition $\delta(a, [s]) = [a :: s]$, and output $o([s]) = h(s)$. Stability makes the transition well-defined; the empty continuation makes the output well-defined. Induction on s proves that the fold returns $[s]$. $\square$

This is a semantic characterization, closely related to the classical fold and automata viewpoints [18, 19]. It is not an effective method for deciding the universally quantified relation (3), nor an extraction of a concise Lean type from arbitrary examples.

A terminating finite construction

Theorem 3.2 (Minimum finite machine consistent with finite samples). *Let A and B be finite explicitly enumerable sets with decidable equality, with B nonempty. Given a finite consistent sample set $D \subseteq \text{List}A \times B$, a terminating algorithm can construct a minimum-cardinality deterministic right-fold machine consistent with D . Its carrier can be represented as $\text{Fin}q$.*

Proof. Let S contain the empty list and every suffix of every sample input. Construct states $S \cup \{\perp\}$, start at $[]$, and set $\delta(a, s) = a :: s$ when $a :: s \in S$, otherwise $\perp$; set $\delta(a, \perp) = \perp$. A sample input never leaves S while being processed from right to left. Give each sample state its specified output and every other state a fixed $b_0 \in B$. Consistency ensures that the assignment is well-defined. This machine has at most $|S| + 1$ states and realizes the samples.

For $q = 1, \dots, |S| + 1$, enumerate every transition table $A \times \text{Fin}q \rightarrow \text{Fin}q$ and output table $\text{Fin}q \rightarrow B$, fixing initial state 0. Fixing it loses nothing because a machine's states can be renamed. Each sample check terminates. The first successful q is minimal; the constructed upper bound ensures termination. $\square$

The number of raw tables at size q is $q^{|A|q}|B|^q$. This construction proves decidability, not ten-second tractability. SAT-style constraints, partition refinement, symmetry breaking, and incremental counterexamples can improve enumeration without changing the finite specification. A Lean-only initial implementation can use direct backtracking over tables with early sample propagation rather than importing an external solver.

This theorem minimizes consistency with the given samples. It does not identify the unknown intended function outside them. Active-learning algorithms obtain additional leverage from membership or equivalence queries; an arbitrary finite contract does not supply those oracles [19].

What changes for a fixed Lean type grammar

For a grammar $\mathcal{K}_b$ bounded by type syntax size, there may still be infinitely many inhabitants of each candidate type. Bound the term grammars for the seed, step, finishing function, proof obligations, constants, and universe instantiations as well. If the resulting space is finite and every validation is a terminating decision, exhaustive enumeration returns the minimum accepted tuple or establishes absence *in that finite*

space. When a validator can return unknown, the result is instead “exhausted with unresolved checks,” not a proof of absence.

Without a term bound, even the finishing obligation $\text{Unit} \rightarrow T$ contains arbitrary inhabitation of T . Thus the broad question cannot be reduced to finite automaton minimization merely by bounding the syntax of K . The practical task is a cost-bounded synthesis problem with a stated grammar, not a universal minimal-type oracle.

Safe sample-based exclusion

Build a graph on observed suffixes. Add an edge $s \# t$ only when a common continuation u has known, different outputs for $u \# s$ and $u \# t$. Any realization colors this graph by its reachable states, so

$$\chi(G_D) \leq |\text{Reach}(K)| \leq |K| \quad (5)$$

for finite K . A witnessed clique is a cheap lower bound; computing the exact chromatic number is optional. No inference of difference is licensed by a missing observation. Compatibility of incomplete rows need not be transitive, so greedily merging every compatible pair is also unjustified.

This excludes `Bool` when three pairwise distinguishable states are forced. It does *not* exclude `Nat` or `List A` because a preliminary enumerator found only two values. A restriction to a finite reachable subset requires a proved invariant covering every allowed program in the bounded class, not a count of values found so far.

Example 3.3 (Boolean output can require three states). On a one-letter alphabet, prescribe $h(a^n)$ to be true precisely when 3 divides n , for $0 \leq n \leq 8$. For suffix lengths 0, 1, 2 and continuation lengths 0, 1, 2, the rows are (1, 0, 0), (0, 0, 1), and (0, 1, 0). Hence two states are impossible, despite the Boolean output type. A three-state cycle realizes the samples. The accompanying script exhaustively checks all 2, 16, and 216 transition/output tables for one, two, and three states, respectively.

3.2 R2.Q2. What is the correct polymorphic alphabet?

Answer. For genuinely parametric container transformations, shapes and input positions give a precise answer. Equality patterns become relevant only when the admitted operations can observe equality. These are distinct semantic regimes, not interchangeable encodings of all Lean polymorphism.

The 2024 paper on example-based realizability of polymorphic programs directly studies this connection through containers, making it particularly relevant to Leant2’s question [17]. The following argument states the portion needed here explicitly.

Theorem 3.4 (Shape–position representation). *Let $F(A) = \sum_{s \in S} (P_s \rightarrow A)$ and $G(A) = \sum_{t \in U} (Q_t \rightarrow A)$ be containers. A natural transformation $\tau : F \Rightarrow G$ is determined by a shape map $\phi : S \rightarrow U$ and, for each s , a position map $\psi_s : Q_{\phi(s)} \rightarrow P_s$, with*

$$\tau_A(s, k) = (\phi(s), k \circ \psi_s). \quad (6)$$

Conversely these data define a natural transformation.

Proof. Evaluate τ on the generic input (s, id_{P_s}) at element type P_s . Its output is a pair $(\phi(s), \psi_s)$. For arbitrary $k : P_s \rightarrow A$, naturality along k says that applying τ to (s, k) equals mapping k over this generic output, giving (6). Conversely, composition of position maps with an element map commutes by associativity of function composition, proving naturality. $\square$

For lists, the input shape is a length n and its positions are Finn . Consequently one generic list of distinct position tokens determines the behavior at each fixed length, assuming naturality. The output may reorder, duplicate, or discard positions; its length is also determined by the input shape. This theorem

does not reduce unbounded list lengths to a finite set of tests. Agreement at lengths at most N proves no agreement at length $N + 1$. For multiple input containers, use disjointly tagged position sets so that provenance from different arguments is not accidentally identified.

A lawful equality operation allows a program to distinguish repeated elements from distinct ones. Such functions need not commute with arbitrary maps that collapse two values. They may instead respect the relevant injective maps or isomorphisms. At bounded input lengths, partitions of positions encode equality patterns; there are finitely many such partitions, but more than a single all-distinct pattern. An arbitrary BEq instance need not even be lawful, so its operational behavior cannot be replaced with mathematical equality without additional evidence.

Finally, a Lean type signature alone does not establish the required free theorem for every admitted provider, classical construction, or type-sensitive operation. A safe implementation should attach relational or naturality certificates to the supported fragment and its providers. Without such a certificate, generic-position tests remain useful heuristics and explanations, not a license for permanent equivalence pruning.

3.3 R2.Q3. Can fusion be run backward to discover auxiliary functions?

Answer. Yes as counterexample-guided lifting and constrained synthesis. There is substantial related work; completeness must be restricted to the chosen lifting language and oracle assumptions.

AutoLifter synthesizes auxiliary summaries and combinators for a family of algorithmic paradigms, including divide-and-conquer, using decomposed synthesis obligations. Its necessary conditions can leave unrealizable subproblems, so its practical decomposition is not a general complete minimal-carrier method [15]. Synduce’s recursion synthesis with unrealizability witnesses shows another useful direction: witnesses can explain why a proposed recursive family lacks sufficient information. That paper is not a blanket theorem of automatic auxiliary-type invention from sparse examples [16].

For Leant2, synthesize a lifted semantic summary H and an algebra together. For a right fold, the desired equations are

$$H([]) = z, \quad H(a :: s) = \delta(a, H(s)), \quad o(H(s)) = h(s). \quad (7)$$

For a concatenation homomorphism, replace the middle equation with $H(x ++ y) = H(x) \otimes H(y)$. Candidate summaries can be tuples of target results and auxiliary features. A counterexample in which identical proposed summaries require different next outputs demonstrates a missing distinction. Use that witness to rank a richer summary, then search its algebra. Keep prior summary alternatives available: locally satisfying a necessary condition does not ensure that the rest of the algebra is definable in the step grammar.

There is a useful completeness statement for a finite feature library. Suppose a finite set of candidate features and a finite algebra grammar have been specified, all their required checks terminate, and the intended summary and algebra occur in these spaces. Fair enumeration of feature tuples and algebra terms is complete for this class. Witness-driven prioritization preserves that claim only when it excludes candidates with valid reasons and does not irrevocably commit to one feature tuple. This is a realistic theorem to formalize; completeness for all polynomial-functor catamorphisms is far broader.

Worked correction: reversal and function carriers

Reversal is already a right fold with carrier ListA:

$$\text{reverse}(s) = \text{foldr } (\lambda a \ r. r ++ [a]) \ [] \ s. \quad (8)$$

The induction proof is the defining equation $\text{reverse}(a :: s) = \text{reverse}(s) ++ [a]$. Thus failure to synthesize this program with a restricted step grammar is not a semantic impossibility of the carrier. The difficulty

may be that snoc is unavailable, too deep to synthesize, or too expensive when implemented by repeated append.

A function carrier gives a different construction. Define

$$D([]) = \text{id}, \quad D(a :: s)(r) = D(s)(a :: r). \quad (9)$$

Then $D(s)(r) = \text{reverse}(s) ++ r$ by induction on s , using associativity of list append in the step. In particular $D(s)([]) = \text{reverse}(s)$. Under an ordinary linked-list cost model, the snoc fold traverses a growing intermediate list, whereas the accumulator construction performs one cons per input element and one final traversal of the closure chain. These are different implementations, not a proof that one type can express reversal and the other cannot.

Worked lifting: maximum prefix sum depends on the scheme

For integer lists, let $h(s)$ be the maximum prefix sum, including the empty prefix. A single right fold needs only an integer:

$$h([]) = 0, \quad h(a :: s) = \max(0, a + h(s)). \quad (10)$$

But the same result alone is insufficient for combining arbitrary adjacent segments. The lists $[0]$ and $[-1]$ both have result 0, while appending $[1]$ gives results 1 and 0. An algebra receiving only the two segment results cannot distinguish those cases.

Lift to $H(s) = (\text{sum}(s), h(s))$ and define

$$(s, m) \otimes (t, n) = (s + t, \max(m, s + n)). \quad (11)$$

Every prefix of $x ++ y$ either lies in x or contains all of x and a prefix of y , proving $H(x ++ y) = H(x) \otimes H(y)$. The reachable invariant is

$$I(s, m) : \quad 0 \leq m \wedge s \leq m. \quad (12)$$

It is essential when stating the monoid laws with identity $(0, 0)$.

Proposition 3.5 (Invariant-restricted summary monoid). *Operation (11) is associative on $\mathbb{Z}^2$, preserves I , and has two-sided identity $(0, 0)$ on the subset satisfying I .*

Proof. Both bracketings have first coordinate $s + t + u$ and second coordinate $\max(m, s + n, s + t + p)$. If both operands satisfy I , the result's second coordinate is nonnegative because it is at least m , and is at least $s + t$ because $n \geq t$. Left identity gives $(s, \max(0, m)) = (s, m)$ using $m \geq 0$; right identity gives $(s, \max(m, s)) = (s, m)$ using $m \geq s$. $\square$

On unrestricted pairs, left identity already fails at $(1, -1)$. A synthesizer that proves algebra laws for all syntactic carrier values instead of reachable ones may reject a correct lifted program. Conversely, assuming an invariant without proving its preservation can certify an invalid decomposition. This example shows why carrier type, recursion scheme, and invariant should be searched as related objects, and why “minimal carrier” is not scheme-free.

3.4 R2.Q4. Is there a complete practical motive enumeration?

Answer. There is a complete finite enumeration for each explicit finite template family, and a fair semidecision procedure over increasing families. There is no reason to expect one small fixed family to contain every useful accumulator or strengthened result.

For a recursor with result type T , begin with motives obtained from the existing result and relevant context: $T, S \rightarrow T, T \times U, \text{Option}T$, and indexed versions that abstract the recursor's indices. Search both the motive and its finishing map; do not count a carrier as solved merely because the recursor application

elaborates. For primitive recursion with parameters, the function-valued motive explicitly retains those parameters. A paired result can make Fibonacci a one-step recursion; this is different from admitting unrestricted nested induction on every natural-number local.

Bound motive syntax, number of extra parameters, tuple width, and the number of active recursion frames. Enumerate by a fixed structural cost, with failed index constraints and observed behavioral distinctions influencing priorities. A precise completeness claim is then: every accepted program in this scheme-and-motive grammar is eventually visited by the fair baseline, assuming terminating validation. It is not completeness for all Lean programs that happen to be primitive recursive.

3.5 R2.Q5. Do theorem-prover generalization heuristics transfer?

Answer. They transfer well as proposal generators. They do not automatically supply sound carrier exclusion, a correct invariant, or a complete synthesis strategy. Theory-exploration systems such as Hipster demonstrate that testing candidate equations and then proving useful lemmas can support inductive reasoning; the reviewed system is an Isabelle integration, not a Lean component that can simply be imported [20].

The relevant information in a failed synthesis is the difference between what the recursive hypothesis provides and what the step requires. For example, trying to prove $D(s)([]) = \text{reverse}(s)$ is too weak to justify a step that invokes $D(s)$ at $a :: r$. Generalizing to all r produces precisely the invariant in (9). A missing recursive summary component suggests tupling; a hypothesis specialized to an unchanged parameter suggests generalization; a repeated contextual fragment suggests a helper.

Record these failures structurally, not just as tactic error strings. A useful record includes the required recursive application, the available hypothesis, unmatched indices or parameters, and residual examples for the missing hole. Generate a few dependency-correct generalizations, attach fresh proof obligations, and resume the same joint search. If a guessed lemma or helper fails, backtrack its consumers as well. Testing a conjectured generalization is a cheap rejection filter; only a checked proof permits it to become a trusted lemma or pruning rule.

4 R3: Evaluation, hole closures, and safe refutation

Proposal source: sections/07-evaluation.tex in [2].

4.1 R3.Q1. Does live bidirectional evaluation already handle higher order?

Answer. Yes in a relevant implementation: Smyth explicitly supports higher-order function examples, while simplifying the formal core to a first-order presentation. It also addresses recursive synthesis without trace-complete examples. Thus “Smyth is first-order and requires complete recursive traces” is not an accurate premise for planning this extension [12]. Neither fact gives a ready-made evaluator for arbitrary dependent Lean expressions.

The next boundary is not simply “a hole contains a function.” It is a *typed contextual closure*. If a hole was declared in telescope Γ_h with type T_h , and evaluation reaches it in context Δ , its closure must contain a well-typed substitution $\sigma : \Delta \rightarrow \Gamma_h$. The residual value has type $T_h[\sigma]$. Two uses of the same dependent hole may therefore produce different indexed types even though they share the same underlying unknown. Store the telescope, substitution, and expected type, not just a hole identifier and an untyped array of values.

Higher-order examples usually specify finitely many applications. They do not give a complete extensional value for a function. A closure-level observation should say that this hole, under this environment, applied to these arguments, must satisfy a typed relation. For example, a result in `Vec A n` cannot be compared with one in `Vec A m` merely because their erased lists agree. The relevant index

equality or heterogeneous relation must remain explicit.

Backward propagation also need not yield a single required output. Inverting an unknown guard creates alternatives; inverting a noninjective function gives a relation or a set of possibilities. An observation such as $f(0) = f(1)$ does not license inventing independent output values for the two calls. Preserve shared constraints between calls. Unevaluation should therefore produce a constraint graph, with deterministic point examples as one useful special case.

A manageable initial fragment includes first-order algebraic outputs, applications of typed holes, known constructor cases, and registered inversion rules carrying proofs of necessity. Unsupported dependent elimination returns unknown and falls back to the existing path. This is useful progress without requiring a second implementation of every Lean conversion rule.

4.2 R3.Q2. What can be reused for assignment-driven memoization?

Answer. Demand-driven incremental computation is a close architectural precedent. Adapton records dependencies and reuses computations when the data they depend on have not changed [21]. The reviewed literature does not supply a verified, drop-in normalizer for all Lean 4.34 expressions with metavariable assignment, rollback, and the proposed completion guarantee. The correct answer is therefore an adaptation of established dependency techniques, not a claimed existing package with all these properties.

Each evaluated observation should carry the versions of the assignments it actually read. A conceptual cache key is

$$(E, \Gamma, e, mode, \{(m_1, v_1), \dots, (m_k, v_k)\}), \quad (13)$$

where E identifies the environment and relevant options, Γ is a canonical local telescope, e is the expression, and the remaining pairs are the dependency fingerprint. Universe assignments and delayed expression assignments belong in this accounting. The key may be represented efficiently with immutable nodes and hashes, but hash equality alone must not be treated as a proof of expression equality.

Assignments are monotone along a branch but not across rollback. A restored branch must regain its earlier assignment versions, while newly created assignments in a sibling branch receive distinct identities. Reusing a numeric metavariable identifier after rollback must not accidentally reuse a result from an unrelated assignment. The implementation can either keep caches persistent with the search node or maintain a globally shared store keyed by persistent identities and validated dependency fingerprints.

The inspected transaction context intentionally leaves the resource ledger outside snapshots. It also contains an `IO.Ref` for residual blockers [5]. Those are different categories of state. Resource charges should never be refunded; a blocker set is a fact about an observation in a particular assignment state. The recommended redesign attaches each blocker set to that observation and state. This is a cache-invariant requirement, not a reproduced failure of the current implementation.

Cache *unknown* carefully. A result stuck on m can be reused only while the relevant assignment remains unchanged; a timeout is reusable as a cost estimate, not as evidence that the result must remain stuck at a larger budget. A cache must not make the semantics of a negative result depend on which sibling happened to consume the budget first.

4.3 R3.Q3. What is the cheapest trustworthy fast evaluator?

Answer. A restricted evaluator with checked output certificates is a safer first target than reimplementing all kernel conversion. Lean4Lean is relevant as an explicit Lean implementation and verification project for type checking; its paper does not establish that every rule of the repository's later Lean 4.34 kernel can simply be extracted as a ready-verified synthesis evaluator [22].

There are three increasingly ambitious options. The first is to retain kernel reduction but decompose contracts into independently cached observations. The second is an untrusted evaluator that proposes a refutation and supplies enough information for a checked proof to confirm it. The third is a small typed

object language with a verified interpreter and a theorem relating its semantics to an embedding into Lean expressions. All three preserve the final Lean gate; only the latter two can justify pruning based on a custom evaluator.

The first option should be implemented before the others. Split conjunctions, Boolean conjunction checks, and finite-list universal checks through proved normalization lemmas. Keep the original contract and a reconstruction theorem. Do not rewrite an arbitrary BEq check into propositional equality unless the required law is present. Quantified propositions over infinite domains remain symbolic unless a sound reduction applies. Dependency-tracked smaller observations already provide the true/false/unknown vectors needed by ranking and guard synthesis.

Proposition 4.1 (Sufficient certificate for open refutation). *Suppose a typed encoding represents every $\theta \in C(S)$, and a checked theorem proves that its evaluator’s result on any such completion agrees with the observation’s Lean meaning. If evaluation establishes that no represented completion can make the observation true, then deleting S for that observation satisfies (1).*

Proof. Assume a completion in $C(S)$ satisfies the observation. By coverage of the encoding it has a represented completion, and by evaluator agreement that completion makes the encoded observation true, contradicting the certified emptiness result. $\square$

The direction of approximation is important. To reject programs safely, an abstract value set must *over-approximate possible completed behaviors*. If even that set cannot satisfy the observation, no real completion can. An under-approximation of behaviors can miss a satisfying completion and is unsafe for this purpose. One may instead speak of a partial evaluator that under-approximates the set of *provably decided judgments*: it says unknown more often, but never makes an unsupported decision. These are different uses of the word “under-approximation.”

For reduction-only certificates, a substitution-stability lemma is central: each permitted reduction step remains a valid step or equality after any well-typed completion substitution. Constructor discrimination and arithmetic on closed literals are convenient initial rules. An unsupported reduction, ill-scoped residual, resource exception, or unproved side condition must give unknown, never false.

This point deserves attention in the current code: the comment on `instantiatePartial` permits a reduction-only expression whose metavariable local contexts no longer match [3]. That representation may be a useful internal shortcut, but a proof of open pruning safety must explain its connection to well-scoped completions. Merely observing that `Kernel.whnf` returned a Boolean is not a replacement for that connection.

Differential testing on closed expressions is valuable for finding mistakes, but cannot establish the universal open-completion property. Test open terms under multiple completions and rollback sequences as well, then prove the restricted evaluator’s substitution lemma. Also measure certificate construction and checking: a fast speculative evaluation followed by a full recomputation at every refuted node may offer no net speedup.

4.4 R3.Q4. Can top-down angelic recursion terminate completely?

Answer. Yes for a carefully finite search model; not by directly importing Burst’s algorithm-specific argument into a different search organization. Burst is the relevant precedent for allowing recursive calls to take favorable values and then refining the assumptions that made those values possible [13].

Angelic execution means existentially quantifying over a *consistent* oracle, subject to known constraints. Repeated identical calls in the same instantiation must not independently choose contradictory answers. A real recursive program determines one allowed oracle, so failure for every allowed oracle can safely refute a branch if established by a sound check. Success for some oracle is only a necessary condition for a realizable program.

There are two safe implementation modes. In a guidance-only mode, angelic results affect priorities but never delete a branch. In a pruning mode, refutations need a certificate for the universal failure of all allowed oracle answers. A single guessed answer that fails is not that certificate.

Proposition 4.2 (Finite refinement termination). *Suppose a bounded search has finitely many typed candidate versions, finitely many oracle assignments per version, terminating validation, and a refinement step that removes at least one currently considered incompatible version or oracle assignment without removing any valid solution. A fair refinement loop terminates, and returns a solution whenever one exists in this finite space.*

Proof. The number of remaining version–oracle pairs is a nonnegative integer that strictly decreases at every unsuccessful refinement. Thus only finitely many such refinements occur. Safe exclusion preserves every valid solution, and fair terminating validation eventually considers it. $\square$

The finiteness assumptions do real work. Infinite integer outputs, fresh helper invention, unbounded unfolding, and proofs that time out violate them. In those settings use dovetailing and return unknown at a finite deadline. A contract observation $f(v) = w$ can constrain a recursive call only when it is a call to that same function at that same full argument tuple. It does not determine the value of an invented accumulator helper on extra arguments. The next section on recursion develops this distinction further.

5 R4: Indexed families, motives, and transports

Proposal source: sections/08-dependent.tex in [2].

5.1 R4.Q1. Which motive searches are finite and complete?

Answer. Enumerating subsets of finitely many eligible occurrences is finite and complete for that syntactic template class, provided abstraction is capture-avoiding and typing is checked. The class is not closed under arbitrary accumulator introduction, auxiliary results, or new inductive invariants. Those operations can introduce types and binders absent from the original goal.

Given a major premise $x : I \vec{p} \vec{l}$, distinguish mandatory *dependency generalization* from optional *strengthening*. Mandatory generalization abstracts the indices and reverts declarations that depend on them or on x , preserving telescope order. Optional strengthening abstracts additional parameters or enlarges the result. The former is primarily an elaboration problem. The latter remains a search problem.

For the vector-map signature, the useful motive is

$$M(n, x) = \text{Vec } B \ n. \quad (14)$$

The recursive hypothesis then has the correct length-indexed result. No new semantic invariant needs to be invented. For a traversal with an accumulator $r : S$, a possible motive is $M(\vec{l}, x) = S \rightarrow T(\vec{l}, x)$; the recursive call can then use an updated r . For a result that must retain an additional summary, the motive may instead be a dependent pair or a product. Neither is generally produced by selecting a subset of existing maximal index occurrences.

A practical bounded enumerator should work as follows. Normalize only enough to expose the inductive family and its recursor metadata. Construct the mandatory dependent telescope. Generate optional generalization sets in increasing size, with dependency closure. Combine these with a bounded result lifting grammar. For each combination, construct a motive by abstraction, check its sort and binder dependencies, try the recursor prefix transactionally, and retain the generated branches with their substitutions.

If nonvariable index expressions occur, first generalize them to fresh index variables with appropriate equalities, rather than applying a routine that requires variable indices to an incompatible major premise. These equalities become ordinary proof obligations; they are not erased assumptions.

Proposition 5.1 (Restricted motive completeness). *Fix a finite occurrence set, a finite family of dependency-closed context subsets, and a finite result-lifting grammar. Suppose candidate construction and checking terminate. Enumeration of every combination, discarding only ill-typed combinations, terminates and finds every well-typed motive in that specified class.*

Proof. There are finitely many combinations of finite sets. Each is processed in finite time. Every member of the class is generated by definition, and its well-typed members survive the stated filter. $\square$

This deliberately modest theorem is valuable: it defines exactly what an “exhausted motive search” means. To enlarge coverage, increase one bounded family at a time, rather than presenting the theorem as completeness for primitive recursion with all possible parameters.

5.2 R4.Q2. How can transports be normalized without losing meaning?

Answer. Keep typed transport nodes in the program representation. Quotienting proof arguments for ranking is often sensible; deleting arbitrary transports or replacing every equality with an internal setoid is not a general canonicalization procedure.

Consider a local value of type $\text{Vec } A \ (n + m)$ and a goal of type $\text{Vec } A \ (m + n)$. A propositional equality of indices can justify transport, but an erased list representation alone does not type the result. Use a checked equality proof to construct the actual cast. Prefer such examples to identities that may already be definitionally equal under Lean’s chosen recursive definition of arithmetic.

Proof irrelevance means that two proofs of the same proposition need not be enumerated as different computational choices. It does not make the source and destination types of a cast interchangeable. Nor does it imply that an arbitrary transport expression reduces to its underlying term. A ranking normal form can ignore proof size while preserving cast boundaries and their endpoints. A display may suppress routine casts only if the actual retained expression remains available and re-elaborates as claimed.

A useful normalization policy consists of a small proved rewrite collection: identity transport, composition along matching endpoints, and normalization of a selected index-expression fragment. Orient rewrites to a known terminating strategy or bound their application. Store evidence connecting the normalized representation with the original typed expression. An arithmetic proof can justify an equality without becoming an instruction to rewrite in both directions indefinitely.

Heterogeneous equality is useful for stating equalities across indexed types, but does not remove the obligation to construct a term at the required final type. A setoid or quotient can represent an intentionally designed semantic language, but then every constructor and operation needs compatibility proofs. That is a substantial new abstraction layer, not a cheap substitute for Lean’s existing dependent expression language.

For hard deduplication, compare candidates at the same expected type and accept only a trusted definitional-equality check or an explicit equality certificate. For ranking, a coarser proof-insensitive fingerprint is acceptable as a grouping key, provided a collision does not discard an unequivalent candidate. Count transport work separately from computational program size when evaluating performance; a zero ranking weight does not make elaboration or proof production free.

5.3 R4.Q3. What do Canonical and sauto actually do about unfolding?

Answer. They provide evidence for controlled, implementation-specific policies, not one universally optimal normalization regime. Canonical’s 2025 description unfolds aliases hiding function types and reduces relevant definitions and instances for its translation; it also discusses expansion and abstraction problems when translating dependencies down to its supported primitives [9]. This is not evidence that every Leant2 provider should be fully unfolded before search.

The public sauto documentation exposes explicitly listed heuristic unfolding through `unfold:` and eager unfolding through `unfold! :`, with special treatment of certain logical definitions. This is materially

different from unconditionally unfolding every definition in the environment [25]. These are Coq-specific controls, not APIs usable directly in the Lean-only default.

For Leant2, use a staged policy internal to the engine. Start with cheap head normalization sufficient for indexing and introductions. When a mismatch is blocked by a definition, try a limited unfolding of that head. Escalate transparency only for the affected constraint. Treat a library component as a useful atomic head while synthesizing compositions, and unfold it for behavior checking only as demanded. A definition can be excellent as a search component even when its expanded body would be a very poor search rule.

Every normalization cache must include the transparency policy and environment version. Failure under a restrictive policy means “not resolved at this stage,” not proof that the candidate types can never be equal. If the advertised bounded grammar permits terms requiring a later transparency stage, the earlier failure cannot justify their permanent exclusion. These distinctions also make diagnostics meaningful: a failed motive due to an unreduced alias is different from a genuinely ill-typed telescope.

5.4 R4.Q4. Which Lean APIs can be reused now?

Answer. The pinned Lean 4.34 source provides public helpers for the relevant part of motive construction. In `Lean/Meta/Tactic/Induction.lean`, `getMajorTypeIndices` checks the form of indices, `mkRecursorAppPrefix` computes a recursor prefix including the motive, and `MVarId.induction` builds and returns the induction branches [6]. These are concrete library entry points, not proposed API names.

The inspected signatures, with formatting shortened, are:

```
Lean.Meta.mkRecursorAppPrefix
  (mvarId : MVarId) (tacticName : Name)
  (majorFVarId : FVarId) (recursorInfo : RecursorInfo)
  (indices : Array Expr) : MetaM Expr

Lean.MVarId.induction
  (mvarId : MVarId) (majorFVarId : FVarId)
  (recursorName : Name)
  (givenNames : Array AltVarNames := #[]) :
  MetaM (Array InductionSubgoal)
```

The helper abstracts the major premise and indices from the current target. The full induction operation manages reversion, introduction, and a `FVarSubst`. Its `InductionSubgoal` includes fields and substitution information. Leant2 should consume that information rather than infer branch locals by positional guesses.

The implementation also explains important failure cases: indices are expected to be free variables, and repeated or improperly dependent index arrangements are rejected. Therefore wrap this API with a preprocessing step for index expressions and a small, typed motive-template search. Catch ordinary tactic failures transactionally, but propagate interruption and resource exhaustion. Preserve the exact reason category: unsupported index arrangement, motive sort mismatch, missing instance, or resource limit.

Do not promise that failures are always cheap. Unification and instance synthesis inside the helper can still perform significant work. Instrument calls and cap them through the same scheduler used for other proof obligations. Pin these wrappers to Lean 4.34 and run compatibility tests before changing the toolchain; the moving “latest” reference manual is not the version pin. No Lean compilation of these integration suggestions was performed for this report.

6 R5: Recursive-call capabilities and realization

Proposal source: sections/09-recursion.tex in [2].

6.1 R5.Q1. Does a search-time interpreter agree definitionally with Lean?

Answer. Not automatically. A faithfully encoded structural recursor can have definitional computation equations. A general well-founded realization may require propositional equation lemmas; the official Lean documentation explicitly distinguishes these situations [23]. A custom interpreter must therefore prove the agreement appropriate to its realization, rather than assume that `rfl` will relate every recursive equation to a `WellFounded.fix` expression.

A capability should be scoped to the current recursion frame and have the fully dependent shape

$$rec_x : \prod_{y:X} r(y, x) \rightarrow B(y), \quad (15)$$

where r is a chosen well-founded relation. Using a nondependent result type in this interface would exclude precisely some of the indexed programs R4 is meant to support. The capability cannot escape into the ordinary provider set as an unrestricted provisional definition of the function being synthesized. Its arguments and decrease proof must remain visible to the realization step.

For a body functional

$$F : \prod_{x:X} \left(\left(\prod_y r(y, x) \rightarrow B(y) \right) \rightarrow B(x) \right),$$

well-founded recursion produces a function f satisfying the appropriate unfolding equation. A search-time interpreter should execute this body only on calls admitted by the capability. It can record a finite derivation tree whose nodes give the current arguments, the selected body branch, and the smaller recursive calls.

Proposition 6.1 (Interpreter simulation by well-founded induction). *Suppose the realized function f satisfies its certified unfolding equation for F . Suppose an interpreter uses the same body semantics and makes only calls justified by $r(y, x)$. If each nonrecursive interpreter operation agrees with the corresponding Lean operation, every completed interpreter evaluation at x returns $f(x)$, with equality interpreted at type $B(x)$.*

Proof. Use well-founded induction on x . For each recursive call at y with $r(y, x)$, the induction hypothesis identifies its completed result with $f(y)$. Substituting these equalities into the agreed nonrecursive body semantics gives $F(x, \lambda y. h.f(y))$. The unfolding equation identifies this with $f(x)$. $\square$

The proof is propositional unless the particular encoding gives stronger conversion properties. A fuel-bounded interpreter can exploit this theorem for successful evaluations if it records the required well-founded call justifications. Running out of fuel proves nothing about the result. Likewise, a branch rejected by an approximate interpreter needs the stronger all-completions property from Section 4; simulation of already closed programs alone does not justify open pruning.

There are two reasonable realization paths. One passes a closed, well-typed recursive equation to Lean's definition elaborator and retains the generated equation lemmas and checked declaration. The other uses a small collection of already certified recursion templates and directly builds their terms. The second may reduce per-candidate elaboration cost, but should begin with a few well-tested schemes, not a parallel implementation of all termination analysis.

Compiled evaluation can be an excellent speculative speed signal. Under the core trust discipline, it is not automatically a proof of the contract. The inspected provider blacklist excludes native reduction axioms such as `Lean.ofReduceBool` [3]; a design that quietly uses them to certify the winning candidate would change that trust policy. Prefer equation lemmas, verified interpretation, or a reconstructed proof checked under the original permitted-axiom profile.

6.2 R5.Q2. Does angelic execution compose with top-down synthesis?

Answer. Yes, but the state must include the constraints on oracle calls, and the final candidate must realize them. Top-down search loses the immediate availability of the completed bottom-up components and finite automata used by a particular bottom-up algorithm. It gains contextual type information and can create highly specific dependent holes. Neither direction dominates without an empirical comparison.

Attach each assumed call to its full typed argument tuple, recursion frame, universe instantiation, and local environment. When the body becomes more precise, replay the affected calls or refine their constraints. A single call may constrain several output observations, so assumptions cannot be maintained as independent per-example values. Use a persistent call-constraint graph shared by the relevant search node, analogous to the metavariable dependency graph in R1.

Contract-derived oracle answers and guessed oracle answers have different logical status. From a proved implication $C(f) \Rightarrow f(v) = w$, one may reason under the assumption $C(f)$ that this particular recursive call returns w . If that reasoning yields a contradiction for every completion of the current sketch, it proves that no completion satisfies C . It does not prove that the call literally returns w in every candidate program.

For an invented helper $g(s, a)$ implementing an accumulator, observations about $f(s) = g(s, a_0)$ normally determine only the distinguished seed a_0 . Reusing those values at arbitrary a is invalid. The helper's generalized invariant, its own derived examples, or later concrete evaluation must supply the missing information. This is a central reason to combine carrier invention with proof and residual constraints rather than treating it as a purely syntactic type choice.

The minimal initial implementation should use guessed angelic values only for ordering. A later certified pruning mode can implement the finite refinement argument of R3. Its fallback enumerator must remain fair, and refinements must be able to reopen a helper or carrier alternative whose initial necessary conditions were misleading.

6.3 R5.Q3. Is there a canonical small recursion basis with known coverage?

Answer. There is no measured minimal basis established here for the particular Leant2 corpus and its core-only provider environment. A sensible small engineering vocabulary can nevertheless be specified and tested. Its expressibility and searchability must be measured separately.

The first tier should contain structural recursion on one selected data argument, with a bounded product- or function-valued result. This covers ordinary folds, paired state such as the two successive Fibonacci values, and many accumulating traversals. The second tier should permit structural recursion while other arguments are generalized and pattern-matched. For example, `zip` can recurse on its first list while taking the second list as a parameter; it does not intrinsically require a new simultaneous-recursion primitive. The third tier should permit a well-founded measure on a tuple of arguments, with a checked decrease obligation at each call.

This third tier supports patterns such as `merge` using the sum of remaining lengths, and Euclidean-style recursion using a measure on the appropriate argument. The arithmetic facts establishing decrease still need proof; naming a measure does not supply them. Mutual, nested, and more general course-of-values schemes should be later extensions with their own typed realization tests.

For each benchmark, first write a witness in the proposed scheme vocabulary and kernel-check it. This establishes *expressibility*. Next search with the scheme and carrier supplied, then with only the scheme supplied, then with neither supplied. These three runs separate body-search difficulty, carrier invention, and scheme choice. Report runtime and proof-checking cost separately. Only the final track answers whether the synthesizer can discover the program within ten seconds. A hand-written witness is not a performance result.

7 R6: Universal contracts and program–proof interleaving

Proposal source: sections/10-contracts.tex in [2].

7.1 R6.Q1. How should value holes and proof holes be interleaved?

Answer. Use dependency-sensitive readiness and staged proof attempts, not a universal rule that proofs must come last. Refinement-type synthesis supplies an important model for using logical requirements during construction, but Leant2 need not import an SMT solver to adopt a finite logical abstraction [26].

A proof obligation can be useful while a program still has holes. Equality constraints may determine an index; constructor disjointness may reject an impossible branch; a proof of an arithmetic side condition may enable a recursive-call capability. In particular, attempting reflexivity for an appropriate equality involving an index metavariable can instantiate that index and substantially reduce the value search. Conversely, a large induction about a mostly unknown body is often premature. Readiness is therefore a property of the obligation’s dependency footprint and available decision procedure, not of its being in Prop.

Represent a proof job by its goal, metavariable dependency set, origin, allowed solver tiers, previous attempts, and any values it may constrain. Run cheap normalization, unification, and direct constructor facts immediately. Schedule a larger proof attempt when the relevant computation is stable enough for its selected solver. If it assigns a shared variable, treat that assignment as a normal transactional branch choice, visible to every dependent goal.

For a recursive specification $R(x, f(x))$, the result can be constructed in a subtype with motive

$$M(x) = \{y : B(x) \mid R(x, y)\}. \quad (16)$$

Each branch obtains recursive values together with their established properties. This makes many local obligations small. It does not ensure that R itself is inductive enough: an accumulator or a relational property may require a stronger R' , followed by a checked implication $R' \Rightarrow R$. Allow failure of a branch proof to reopen the invariant and carrier choice.

A Lean-native analogue of a finite qualifier abstraction can maintain predicates from a fixed vocabulary, such as equalities of lengths, bounds on indices, and registered membership or order relations. Candidate invariants are conjunctions or selected combinations from this finite vocabulary. Check each demanded implication using a proof-producing local procedure. “Not proved” remains unknown unless a counterexample or other refutation is obtained. The method trades completeness for a stated finite predicate language, while keeping the logical meaning of every accepted implication exact.

The acceptance gate must audit both the program and its contract proof. Passing a finite collection of test instances may prioritize proof jobs, but it cannot change the universal request into a finite observation contract.

7.2 R6.Q2. Can the easy induction class be recognized?

Answer. A useful sufficient class can be recognized by a checked proof-template procedure. A complete cheap syntactic recognizer for all propositions admitting some successful “induct, simplify, arithmetic” proof should not be promised. Its boundary depends on available lemmas, normalization strategy, invariant choices, and the arithmetic fragment.

Define a template class relative to explicit data: a structural recursion scheme, a predicate template R , a terminating normalization strategy, a finite set of certified component lemmas, and a complete decision procedure for a chosen residual logical fragment. The class contains tasks whose base and step obligations, after inserting the induction hypotheses and performing the specified normalization, lie in that residual fragment and are valid there. If all these checks succeed, the assembled induction proof is a certificate of membership and correctness. Failure gives no claim about another invariant or another proof strategy.

Proposition 7.1 (Local inductive certification). *Suppose a structurally recursive function is described by a fixed constructor algebra. For each constructor, suppose a checked branch proof derives the postcondition for that constructor from the postconditions of exactly the recursive calls made by the algebra. Then structural induction assembles a proof of the postcondition for every input.*

Proof. Apply the induction principle of the input datatype. In each constructor case, the induction hypotheses supply precisely the assumptions required by the corresponding checked branch proof. The resulting constructor cases establish the universal postcondition. $\square$

Length preservation for a map-like traversal is a typical candidate: the recursive hypothesis supplies the length equation for the tail, and the step reduces to constructor arithmetic. A proof about an accumulator typically needs an equation quantified over the accumulator, not just its distinguished seed. A sorting specification needs algebraic facts connecting insertion, ordering, and permutation; those facts do not emerge from arithmetic alone. A nested property such as $f(f(x)) = x$ may require additional lemmas relating two traversals. These distinctions should be benchmark labels rather than hidden exceptions inside a purported universal induction tactic.

The recognizer should return the failed local obligation when it cannot finish. That obligation becomes input to the generalization proposer in R2, the lemma retriever in R7, or a bounded proof job. This makes the failure diagnostically useful without overstating what was decided.

7.3 R6.Q3. Which Lean induction automation is usable?

Answer. The most dependable default is to combine a small explicit induction planner with Lean’s existing local tactics. The pinned source gives a reusable `MVarId.induction` operation, as discussed in R4. The current tactic reference also documents `fun_induction` for induction principles associated with recursive functions; it is useful when the proof should follow the function’s recursive equations rather than a guessed datatype variable [6, 24].

The same documentation presents `grind` as general equality, instantiation, and theory automation. It should not be treated as a promise of arbitrary induction or invariant invention [24]. A workable architecture is therefore “choose and apply an induction principle, normalize the local case, then invoke bounded arithmetic or equality automation.” The induction planner and the local solver have separate resource budgets and separate failure reasons.

Aesop, Canonical, and the Lean hammer are valuable comparisons and possible optional integrations, but their existence does not establish the latency or coverage of a core-only induction service for Leant2’s synthesized programs [8, 9, 30]. Theorem-exploration techniques from other proof assistants are methodological precedents, not evidence that a production rippling-and-lemma-invention library is already available in the pinned Lean core. No such complete ready-made facility was established by the sources reviewed here.

Implement the initial service with explicit calls to the pinned meta APIs and a small portfolio drawn from available imports: reflexivity, simplification, natural-number arithmetic, and general equality reasoning. Preserve generated function equations as named, checked facts when realizing recursive programs. Names of moving-reference tactics should be validated in the Lean 4.34 build before they are treated as stable API dependencies. This report verified the induction source interface statically, not by compiling a new tactic.

A cooperative tactic limit should account for heartbeats or other supported logical resource controls as well as elapsed time. If a solver or foreign process does not honor cooperative cancellation, a hard wall-clock guarantee requires a process boundary. Catching a normal tactic failure is not a general mechanism for forcibly stopping arbitrary computation. This distinction should be reflected in the service’s timeout contract.

7.4 R6.Q4. Is there a principled proof-debt scheduler?

Answer. Yes as an explicit resource-allocation policy with fairness and measured utility; an optimal bandit policy is not justified without a suitable stochastic model. These jobs are correlated through shared metavariables, counterexamples, new lemmas, and changing program bodies. A theorem for independent stationary arms does not automatically apply.

Treat a proof job as yielding one of three useful events: a certificate, a checked counterexample or contradiction, or information that changes another job’s readiness. Let estimated benefits be V_{cert} and V_{prune} , with estimated success probabilities for the next tier. A practical priority score can be

$$U(j) = \frac{\widehat{p}_{\text{cert}}(j)V_{\text{cert}}(j) + \widehat{p}_{\text{prune}}(j)V_{\text{prune}}(j)}{\widehat{c}(j) + \widehat{c}_{\text{switch}}(j)}. \quad (17)$$

This is a heuristic definition, not a claimed optimality theorem. It makes the tradeoff measurable and can be learned from the engine’s own event log.

The deterministic baseline should not depend on a learned score. Use weighted round-robin or deficit scheduling between synthesis, refutation, local proof, and global proof queues. Reserve a positive work share for each nonempty eligible queue and increase each job’s local allowance geometrically after an inconclusive attempt. This prevents the proposal’s “only when the rest of the search is idle” rule from starving proof jobs in a search that is never idle.

Before an expensive proof tier, cheaply replay known counterexamples and small test instances. Do not require testing before zero-cost logical propagation that may itself determine the candidate. Avoid repeated proof attempts after irrelevant assignments by using the dependency versions from R3. Maintain a small certification reserve near the deadline so that a promising completed candidate is not found with no time left to check it. This improves opportunity, not a guarantee of certification under every deadline.

The receipt should distinguish search work, proof work, cancellation, and unresolved debt. A surviving candidate without a proof of the requested contract is “unrefuted, not certified.” If only certified answers are exposed by the frontend, retain such survivors internally for continuation rather than printing them in the verified result list.

8 R7: Retrieval over dependent and class-polymorphic libraries

Proposal source: sections/11-retrieval.tex in [2].

8.1 R7.Q1. Which type abstraction is useful and conservative?

Answer. Use a layered abstraction that preserves enough binder relationships to avoid immediate collapse, and represent applications as AND-hyperedges. Type heads alone are a useful first index, not a sufficient semantic model of component composition.

For a declaration

$$c : \prod_{x_1:T_1} \cdots \prod_{x_n:T_n(x_1,\dots,x_{n-1})} R(x_1, \dots, x_n),$$

its abstract transition must account for *all* required arguments, their shared variables, and the result. An ordinary graph edge from any one argument head to the result forgets the other prerequisites. Such a graph can be a coarse over-approximation, but its paths should not be advertised as realizable compositions. An AND-hyperedge states the prerequisites more faithfully.

A first refinement layer retains rigid type constructors, arrow structure, parameter-variable sharing, and a wildcard for unresolved indices. A second retains constructor patterns of selected indices and simple equalities between them. A third retains a limited universe-constraint skeleton and class requirements. If

some information is deliberately forgotten, use an explicit unknown value whose meaning includes all possibilities. Do not turn an unknown universe level or unsolved index into an incompatibility.

For example, retaining that two list arguments have the *same* element type rules out more spurious paths than recording two independent `List_` nodes. Retaining that the result length of vector concatenation is a sum distinguishes it from length-preserving map without deciding every arithmetic equality in advance. Only query-relevant indices need this additional precision.

Proposition 8.1 (Reachability as a safe exclusion test). *Suppose every admitted concrete construction step maps to an abstract transition and every admitted starting term maps to an abstract initial fact. If the abstract system cannot derive any abstraction containing the target within an explicitly corresponding construction bound, no concrete derivation within that bound exists.*

Proof. Map a hypothetical concrete derivation step by step into the abstract system. The hypotheses give an abstract derivation from initial facts to an abstraction of the target, contradicting abstract nonreachability. $\square$

The hypotheses are the hard part. Include constructors, local reuse, projections, lambda introduction, relevant type conversion, and the allowed recursion templates. Lean permits contraction: using a local value twice does not consume it as a linear resource. An imported Petri-net encoding must make this explicit. A prepass restricted to paths of length three is an optimization for that bounded composition class, not a proof that a four-component solution does not exist.

Use concrete checking failures to refine abstractions only when the failed constraint is understood. A timeout, an unresolved higher-order metavariable, or an instance search stopped at a cap is not a type incompatibility theorem. Keep a fair low-priority path for providers excluded only by heuristic relevance; reserve hard exclusion for a conservative abstraction with an explicit grammar scope.

8.2 R7.Q2. Has type-guided abstraction refinement handled type classes?

Answer. Yes. Hoogle+ explicitly incorporates type classes through dictionary passing and adds corresponding dictionary-producing components and constraints to its abstract search. Its treatment is described in the overview and implementation discussion; it is not limited to type-class-free Haskell [14]. This answers the existence question affirmatively while leaving dependent Lean dictionaries, local instances, and universes as adaptation work.

In the Lean design, model an instance requirement as an ordinary typed obligation for a dictionary. An instance declaration becomes a provider whose prerequisites include its own dictionary arguments. A method use is not complete until the dictionary is supplied. Preserve the full applied class expression, not just a pair consisting of one class name and one type: classes may have multiple parameters, dependent parameters, and output parameters.

Instance caching also depends on context. Use the instantiated class goal, relevant universe information, environment fingerprint, local-instance configuration, and resolution options. A successful ground dictionary term can be reused after compatibility checking. A failure with unsolved parameters must remain conditional on their assignments; a failed or timed-out attempt must not become a permanent negative cache entry for every later instantiation.

Dictionary arguments are not generally proof-irrelevant. A dictionary can contain the addition, order, comparison, or other operation that determines a program's output. Two terms using different computational dictionaries should not be merged because their printed types match. Conversely, successful instance search can determine an output parameter and make a type hole ready. This is another reason to replace rigid deferral rules with the joint dependency model, rather than postponing all instances until every type metavariable is already fixed.

8.3 R7.Q3. Can Lean’s selector take data goals and examples?

Answer. The interface can receive a metavariable whose target is a data type, but that does not establish that a particular theorem-oriented selector will rank programs effectively. The pinned Lean 4.34 interface is [7]:

```
abbrev Selector :=
  MVarId -> Config -> MetaM (Array Suggestion)

-- Relevant Config fields:
maxSuggestions : Nat
caller         : Option String
filter         : Name -> MetaM Bool
hint           : Option String
```

A caller can request a data-oriented filter and can put text in `hint`. The source explicitly allows implementations to ignore that hint. There is no structured, typed input–output observation field in this interface. Passing examples as text is therefore a possible adapter, not a guaranteed behavioral retrieval service.

Define a Leant2-specific query object containing the exact target expression, its local telescope, the current dependency state, typed observations, an allowed-provider policy, and the desired number of proposals. A default implementation can combine the type index, conservative hypergraph reachability, and cheap behavioral features. Existing library suggestions can provide another ranked list, followed by the same concrete type checking. Do not grant retrieved names a bypass around trust restrictions or the demand filter merely because a selector assigns a high score.

The recent literature still makes the domain distinction important. Lean hammer premise selection and the 2026 LeanSearch v2 work concern theorem retrieval; they do not establish accuracy on accumulator invention or data-component retrieval from behavioral examples [30, 31]. A useful result for one task is not a measured result for the other.

A feasible corpus construction

The proposed corpus of library definitions is promising, but “erase the body” is only the beginning. Use a definition’s type and sampled or proved behavioral constraints as the synthesis input; keep its body hidden as a reference for labeling and checking. Record which earlier declarations the reference body uses as one set of relevance labels, without treating those labels as the only possible correct implementation.

Freeze the task environment before the target definition. Exclude the target constant, direct aliases, generated equations revealing its implementation, and later theorems or wrappers that trivially reconstruct it. Split by module or semantic family, not random individual lines. Otherwise a retriever can score well by returning a renamed or theorem-packaged copy of the answer. Public-corpus model pretraining remains a separate contamination risk even after these local exclusions.

Build a core/Std track first so that the default engine remains core-only. An optional ambient-library track can use definitions from whatever the user has imported, but its results must be reported separately. Measure recall of *some sufficient component set*, time spent loading and indexing, and end-to-end certified synthesis within the fixed budget. Reference-body dependency recall alone can punish a valid alternative algorithm and reward a component list whose joint search is far too large.

9 R8: Ranking, equivalence, and stopping certificates

Proposal source: R8 in sections/12-ranking.tex in [2].

9.1 R8.Q1. Is there a defensible objective for the first answer?

Answer. Yes: a declared prior or description-length objective is defensible as a design choice. No source reviewed here establishes that a particular such objective matches the preferences of Leant2’s users. The proposal’s existing ranking keys and any learned replacement should therefore be evaluated as hypotheses, not treated as a discovered universal notion of the intended program.

After enforcing the type, contract, and trust profile as hard requirements, define an explicit score such as

$$J(p) = L(p \mid T, \Gamma, \mathcal{G}) + \lambda_c C_{\text{run}}(p) + \lambda_a A_{\text{aux}}(p), \quad (18)$$

where L is a code length under a stated grammar, C_{run} is a specified measured or certified computational-cost feature, and A_{aux} penalizes invented auxiliary structure. Take nonnegative weights. For reproducible ranking, use a deterministic operation count or freeze any measured cost features in the receipt; freshly measured wall times are not stable mathematical lower bounds. The weights and features can be fixed internally rather than exposed as user settings. A proof-size score may be a separate tie-breaker; making large proofs cheap for display does not make them cheap to find or verify.

A probabilistic grammar gives an ideal code-length interpretation: $L(p) = -\log P_{\mathcal{G}}(p)$, with all productions assigned positive probability in the supported language. Learning grammar probabilities is a well-established synthesis idea, including component guidance and reusable abstraction learning [27, 28]. What is new in a Leant2 experiment would be validating the usefulness of the selected objective for its dependent types and contracts, not the existence of probabilistic ranking.

Unused inputs are useful warning features, but not a mandatory semantic criterion. A correct constant function may intentionally ignore all its inputs. A length function discards element values for good reason. A normal-form occurrence analysis over-approximates possible information flow; it does not decide exact semantic influence. Rank such features softly, and keep the explanation tied to what was actually computed: “argument not syntactically used” is more honest than “argument cannot influence the result” without an appropriate proof.

A strong evaluation combines reference ranks with alternative valid solutions, behavioral diversity, user judgments on underspecified tasks, and the number of examples needed to identify a preferred candidate. Reference rank alone can favor imitation over a simpler or more efficient correct implementation. Report the ranking objective in receipts so that a later algorithm change does not silently change what an “optimal” answer meant.

9.2 R8.Q2. Which candidate equivalence is cheap and meaningful?

Answer. Use a hierarchy rather than one overloaded equivalence relation. At the strongest practical level, compare exact typed terms using a validated definitional-equality check. A proved propositional or extensional equality can merge additional terms. At the lightweight presentation level, group by values on a named finite probe set. Only the first two justify an assertion of equality beyond those probes.

Define finite observational agreement explicitly:

$$p \approx_D q \iff p \text{ and } q \text{ have equal defined observations at every probe in } D. \quad (19)$$

The observations must be typed; a stuck observation should be recorded as unknown, not as an equal result. For dependent outputs, include index information and use a justified equality or transport before comparing values. For functions, a finite set of applications does not prove extensional equality. For type-valued outputs, an empty or unsuitable test set must not collapse unrelated inhabitants such as distinct type constructors.

Presentation groups should retain their members or a reproducible way to regenerate them. When a new counterexample arrives, split the groups. If only one arbitrary representative survives, the retained representative may fail while a discarded candidate satisfies the refined contract. Hard memoization of open subterms requires still more: the equivalence must be a congruence for all contexts in which those subterms may later be used. Merely being a closed term of ground type does not establish this for unknown future observations.

Printed syntax is an especially weak hard-deduplication key because omitted implicit arguments, dictionaries, universes, or casts can hide meaningful differences. Use a canonical expression fingerprint for indexing, followed by an exact check when a collision would discard a candidate. A proof-insensitive fingerprint may save comparisons, but is not by itself the equality certificate.

When can the search stop with an optimality claim?

Let $p_\star$ be the best certified candidate under the fixed objective J . A sufficient stopping condition is a validated lower bound $L(S)$ for every unexplored state and deferred lane such that

$$\inf_{S \text{ unexplored}} L(S) \geq J(p_\star). \quad (20)$$

This includes pending carrier proposals, provider retrieval, proof debt, and alternatives held outside the main queue. Every relevant candidate must be represented somewhere in that accounting. An unexplored grammar extension cannot be silently ignored while claiming optimality over the extended grammar.

The original proposal’s cost-frontier idea is sound only after aligning the frontier lower bound with the *same* objective used to rank final answers. A lower bound on rule depth is not necessarily a lower bound on unused inputs, rendered size, or measured runtime. Learning may change exploration order but must not invalidate the bound. Without (20), report best-so-far. A grace period is a responsiveness policy, not a mathematical optimality certificate.

10 R9: Learned proposals without a new trust dependency

Proposal source: R9 in sections/12-ranking.tex in [2].

10.1 R9.Q1. What accuracy do current models achieve on carriers and helpers?

Answer. The reviewed sources do not provide a directly applicable accuracy measurement for “Leant2 type plus examples to a useful accumulator or helper, within its ten-second core-only budget.” No model experiment of that kind was run for this report. A numerical answer inferred from theorem-proving success, code generation, or premise-retrieval benchmarks would be misleading.

The question can be made experimentally precise. A carrier proposal has at least four success levels: it is syntactically valid, it elaborates as the required type, some program in the downstream bounded grammar can use it to meet the contract, and Leant2 actually finds and certifies that program within the available budget. Only the last measures the interactive benefit. The same distinction applies to a helper signature or a proposed generalized invariant.

Construct a blinded suite from the existing stretch cases plus held-out recursive families. Remove target names and comments that reveal the operation, control the number and informativeness of examples, and keep the allowed providers fixed. Compare an ordinary grammar baseline, a hand-supplied correct carrier, a hand-supplied correct motive or helper signature, and a model proposal lane. The oracle-assisted tracks separate the potential gain from the quality of the model and from weaknesses in the body search.

Record top- k proposal validity, realizability within the frozen grammar, end-to-end certified solve count, total latency including model startup and communication, and effect on cases already solved by the baseline. Stratify by ordinary folds, function carriers, optional state, tupled state, indexed motives, and

helper invention. Split by semantic family to reduce template leakage. A model that proposes the right carrier but consumes the whole budget has not improved the user’s result.

The existing literature on learned synthesis and Lean premise selection motivates such an experiment, but its measurements do not answer this specific question [27, 28, 29]. The honest conclusion is a concrete study design and clear success criteria, not an invented current-model percentage.

10.2 R9.Q2. How can the contribution be deterministic and auditable?

Answer. Make model output an untrusted, typed proposal artifact and separate its generation from its deterministic admission into search. Fixed seeds and temperature zero are not by themselves an implementation-independent replay specification: the model software, numerical execution, tokenization, and request ordering also belong to the generating process.

The durable object should contain the exact request digest, permitted environment, normalized proposal expressions, decision-point identifier, model or generator version, and the policy under which proposals entered the queue. Replay uses those stored proposals, not a new model invocation. A proposed carrier must elaborate as a type; a helper must have a well-scoped telescope; a proof must prove the exact requested proposition. Reject arbitrary commands, new axioms, unsafe declarations, and side-effecting elaboration instructions outside the declared proposal language.

A proposed receipt schema is:

```
Receipt:
repository_commit, lean_version, environment_digest
request_expression, contract_expression, trust_profile
grammar_version, ranking_objective, scheduler_version
normalized_proposals, proposal_provenance
logical_work_budget, completed_checkpoint
admitted_events, unresolved_jobs, stop_reason
accepted_programs, contract_proofs, axiom_audits
```

Environment digests should identify imported artifacts and relevant local instances, not merely the module names. Fresh names should be canonicalized for receipt identity. The inspected gate creates scratch names using a clock [4]; those names need not be semantically significant, but should not become the basis of result hashes or deterministic tie-breaking.

Admit asynchronous proposals at deterministic logical checkpoints, with a fixed ordering of proposal identifiers. Otherwise two runs with identical model outputs but different arrival times can explore different branches. The baseline grammar retains a positive share of work. This provides a replayable policy and preserves eventual baseline coverage under the fairness assumptions of R1; it does not imply identical candidate sets at every wall-clock deadline.

Finally distinguish reordering from extension. A model that proposes an alternative already in the bounded grammar can reorder that grammar. A model that invents a helper type outside it has extended the candidate language. Either can be safe for acceptance, but the latter changes the scope of any bounded negative or optimality result. Record the extension explicitly and include it in coverage accounting. The default Leant2 build can remain Lean-only and model-free; this proposal interface does not require a network service, foreign runtime, or user configuration menu.

11 A unified implementation and research plan

11.1 One joint state, several certified services

The individual answers suggest a smaller conceptual redesign than nine independent research projects. Keep exact Lean expressions and one reversible joint state, but make dependencies and the logical status

of conclusions explicit. Candidate carriers, recursive schemes, motives, helper types, retrieved providers, and learned suggestions are all *proposals*. A proposal enters the same typed constraint process. No service can create an accepted answer or silently turn a timeout into a refutation.

The conceptual interfaces below are pseudocode, not a compiled Lean patch. They illustrate the data that needs to cross service boundaries:

```
JointState:
  requestId
  partialProgram, expectedType, originalContract
  metaState, localTelescopes, universeConstraints
  dependencyGraph, observationStates, recursiveFrames
  proofJobs, proposalFrontier, costLowerBounds
  branchIdentity, assignmentVersions

ObservationResult:
  proved(certificate, dependencies)
  refuted(certificate, completionScope, dependencies)
  residual(typedConstraint, dependencies)
  unknown(reason, dependencies)

Proposal:
  decisionPoint, typedPayload, expectedInterface
  grammarMembershipOrExtension, priority, provenance
```

A Boolean “success” is too weak for many of these interfaces. In particular, a failed instance resolution, unproved lemma, inconsistent finite oracle table, and closed counterexample carry different meanings. Preserving those meanings allows safe caching and useful diagnostics.

A search iteration can then be described without committing to one search strategy:

```
while logical work and wall deadline remain:
  select an eligible job by the deterministic scheduler
  restore its persistent joint state
  run one bounded service action transactionally
  validate and attach any decisive certificate
  invalidate only observations whose dependencies changed
  split or merge dependency components when justified
  enqueue resulting alternatives and proof obligations
  send completed candidates to the request-indexed gate
  update best-so-far ranking and the unexplored-cost ledger
```

The state graph may share immutable structure aggressively. It must not share mutable assignments between incompatible alternatives. Completed certificates and context-independent checked terms can be reused more widely than residual constraints whose types or assumptions remain branch-dependent.

11.2 Suggested module boundaries

The following paths are proposed additions or refactoring targets, not existing files claimed to be present in the repository.

Proposed module	Responsibility
Search/Dependencies.lean	Dependency closure, component interfaces, and branch-safe scheduling readiness.
Search/Agenda.lean	Fair logical-work scheduling and separate heuristic priorities and lower bounds.

Proposed module	Responsibility
Observe/Normalize.lean	Certified decomposition of contracts; retention of the reconstruction proof.
Observe/Incremental.lean	Per-observation typed residuals and assignment-versioned caches.
Observe/Certificates.lean	Restricted checked evaluation and refutation evidence.
Synthesis/Carriers.lean	Bounded type, seed, step, finish, and invariant proposals.
Synthesis/Motives.lean	Dependency-aware generalization and wrappers over pinned induction APIs.
Synthesis/Recursion.lean	Scoped recursive-call capabilities and checked realization.
Retrieve/Components.lean	Type/dictionary indexing, conservative reachability, and optional selector adapters.
Proof/Jobs.lean	Local and global proof jobs, tier escalation, and unresolved-debt status.
Accept/Receipt.lean	Request-bound checking metadata, canonical result identities, and replay.

It is unnecessary to create every module before obtaining a useful improvement. The current gate and transaction discipline already offer important foundations [4, 5]. In particular, keep resource charges monotone across rollback while moving observation-specific blockers into properly scoped state.

11.3 Implementation order and exit criteria

Phase 0: make claims and measurements precise. Freeze the baseline commit and exact imported environment. Add structured outcomes and request-bound gate checks, deterministic candidate identifiers, and logs separating search, evaluation, realization, and proof time. Preserve the original request in every acceptance record. Exit when regression tests exercise timeout, rollback, universe generalization, and axiom rejection without misclassifying unknown as failure.

Phase 1: recover inexpensive lost capability. Implement per-observation decomposition and dependencies, index-aware induction through the existing meta APIs, and context-correct dictionary caching and retrieval. These address several proposal failures without first solving unrestricted carrier inference. Exit when the indexed-vector and missing-lemma cases have checked reference witnesses and the new paths can find them in controlled experiments. Do not announce a speedup until the same harness, environment, and budget have been run before and after the change.

Phase 2: make carrier invention a bounded synthesis problem. Introduce the joint carrier/seed/step/finish grammar, finite-carrier exclusion certificates, and a small feature-based lifting search. Start with sketches whose carriers are supplied, then remove the carrier hints. Add invariants for reachable states when algebra laws require them. Exit when the stretch suite reports expressibility, body-only search, and full invention as separate tracks, including the cost of unsuccessful carrier proposals.

Phase 3: increase recursive and logical reach. Add certified recursive-call capabilities, trace replay, local subtype proofs, and a bounded generalization loop. Only after those are stable should guessed angelic values be used for anything stronger than ordering. Exit when structural and well-founded realizations have independently checked agreement tests and when universal contracts are certified rather than merely tested.

Phase 4: optimize and learn. Use measured traces to improve retrieval, heuristic scheduling, and proposal ranking. Optional learned proposals enter through the same typed interface. Exit criteria include total latency, retention of solved baseline cases, certificate success, and reproducible replay. A large model’s plausibility is not an exit criterion.

These phases deliberately put cache correctness, observation structure, and motive reuse ahead of an ambitious new general-purpose evaluator. Carrier invention remains important, but its strongest pruning and proof feedback benefit from those earlier services.

11.4 Adversarial tests that should accompany the happy-path suite

Table 3: Regression tests motivated by the expert questions.

Test family	Required behavior
Shared index metavariable	Solving one constraint updates all coupled goals; independent-branch memoization does not commit an incompatible index.
Sibling rollback	A residual cached after an assignment in one branch is not reused after a different sibling assignment, even if identifiers are recycled.
Delayed assignments	Observations depend on the pending hole and its typed environment; incomplete instantiation never yields an unsupported refutation.
Incomplete sample rows	Unknown table entries do not create differences or justify transitive merging of compatible rows.
Finite versus infinite carrier	A three-state witness rejects a two-element carrier but not an unrestricted natural-number or list carrier.
Reverse without snoc provider	Failure is reported relative to the step grammar, not as semantic impossibility of a list-valued fold.
Parametricity boundary	A lawful generic-position certificate enables a restricted theorem; an arbitrary equality dictionary or unproved provider does not.
Dependent cast	A vector at a propositionally equal index can be transported with a proof; the cast is not erased from the retained typed term.
Dictionary-sensitive behavior	Candidates differing only in computational dictionaries are not merged through pretty-print equality.
Well-founded evaluation	Successful trace replay agrees with the realized equation; fuel exhaustion is unknown rather than false.
Invented accumulator helper	A seed-specific observation about the target is not treated as a universal specification of the helper’s extra parameter.
Proof debt and CEGIS	New counterexamples split stored probe groups; proof jobs receive work even while synthesis remains busy.
Request binding	A valid proof of an unrelated proposition cannot satisfy a request merely because it was supplied with a valid proof type.
Learned grammar extension	A new proposal changes the recorded language scope; old bounded-exhaustion or optimality claims do not silently extend to it.
Deadline replay	Identical logical receipts reproduce candidates; different wall-clock prefixes are reported with different checkpoints rather than asserted equal.

11.5 What the evaluation should report

For every task, report the target grammar and providers, candidate construction count, residual evaluations and cache reuse, carrier and motive alternatives, proof and realization work, total latency, and the exact stopping reason. Track certified success separately from an unrefuted survivor and from a hand-supplied witness. Report medians and tail behavior as well as totals; speeding easy cases while making a few previously solved queries time out is not a uniformly better interactive tool.

The repository’s historical harness scores are useful baseline evidence, but were not rerun here [1]. The proposal’s ten-second success target and its desired limit on regressions are experimental goals, not results of this investigation. Its highest-value new experiments are the carrier-given versus carrier-invented split, typed-observation cache ablations, and independent measurement of final certification cost.

11.6 Overall conclusion

Leant2 does not need to choose between a completely verified monolithic synthesizer and unconstrained heuristic search. It can use highly speculative proposals while restricting decisive operations to well-scoped, checked conclusions. The kernel protects returned proofs; completion certificates protect pruning; a bounded grammar ledger protects negative claims; a fixed objective and frontier protect optimality claims; receipts protect replay. These are complementary layers.

The expert questions are consequently a mixture of answered literature questions, tractable finite formalization problems, and empirical engineering questions. Reusing the answered parts makes the remaining research more focused: inferring small typed state and adequate invariants from sparse observations, representing dependent residual constraints efficiently, and allocating an interactive budget without losing the exact meaning of a result.

A Executed checks and reproducibility

A.1 What was actually executed

The package includes `experiments/check_constructions.py` and its deterministic `experiments/results.json`. The script uses only Python’s standard library and is compatible with Python 3.9 or later. It was executed while preparing this report. All assertions passed. No Lean executable was available in the working environment, so no new Lean code, proof, tactic, or Leant2 harness run is represented as having been checked here.

The finite checks are intentionally independent of the repository’s performance harness. They validate the arithmetic and operational examples used in the article, not the general correctness of a new synthesizer. The universal mathematical statements depend on the proofs and assumptions in the text, not on extrapolation from these finite runs.

Table 4: Actual finite-check counts.

Check	Count
Binary input lists of length at most eight	511
Reversal as a list-valued snoc fold	511
Function-carrier reversal with binary accumulators of length at most three	7,665
One-state unary Boolean-output tables enumerated	2
Two-state unary Boolean-output tables enumerated	16
Three-state unary Boolean-output tables enumerated	216
Matching tables at one, two, and three states	0, 0, 2

Check	Count
Integer lists over $\{-1, 0, 1\}$ of length at most four	121
Maximum-prefix-sum ordinary right-fold checks	121
Lifted summary checks on pairwise concatenations	14,641
Sampled summary states satisfying the invariant	22
Two-sided identity checks on those states	44
Invariant closure checks	484
Associativity checks on triples of sampled states	10,648

The unary machine search fixes the initial state to zero and exhausts every transition and output table for each reported state count. Its first matching three-state machine has transition vector $(1, 2, 0)$ and output vector $(\text{true}, \text{false}, \text{false})$. This exactly matches the finite lower-bound example in R2; the second matching table is a state-renaming variant. The script separately checks the complete distinguishing rows.

The summary-algebra tests deliberately include a negative check: the unrestricted pair $(1, -1)$ invalidates the proposed identity unless the reachable-state invariant is imposed. Another check demonstrates that maximum prefix sum alone cannot support the chosen concatenation scheme, even though it is sufficient for an ordinary right fold. These checks are designed to expose the two common carrier-inference mistakes discussed in the main text.

A.2 Rebuilding the package

From the package directory, run:

```
python experiments/check_constructions.py
latexmk -pdf -interaction=nonstopmode -halt-on-error \
  leant2-expert-answers.tex
```

The document uses standard TeX Live packages, including `newtx`, `microtype`, `hyperref`, and `xurl`. No remote resources or external bibliography processor are required. With no `latexmk`, run `pdflatex` repeatedly until references and the table of contents stabilize. The PDF supplied in this package was compiled from the included source and visually inspected after rendering.

A.3 Source and uncertainty boundaries

The repository commit and source paths in the bibliography make the investigation revisitable. Official Lean source APIs were inspected at tag `v4.34.0`. The reference-manual links use the moving latest documentation and were consulted on 21 September 2026; they should not be substituted for the source pin when implementing exact API calls. Recent preprints are identified as preprints.

No claim is made that the proposed cache design, proof scheduler, carrier invention procedure, or learned-proposal study already achieves a particular speedup. No negative literature conclusion is a proof that related work does not exist: statements such as “not established by the reviewed sources” identify the limit of this investigation. The concrete recommendations and restricted mathematical constructions remain usable without turning those limits into stronger claims.

References

- [1] Leant2 contributors. *Leant2 repository and README*. Snapshot 784664ff34e819422510a4d470ab07fe48bb736b, 18 September 2026. <https://github.com/VladimirReshetnikov/Leant2/blob/784664ff34e819422510a4d470ab07fe48bb736b/README.md>.

- [2] Leant2 contributors. *Further-improvements proposal*. In particular, sections/04-adaptation.tex, 05-search.tex, 06-carriers.tex, 07-evaluation.tex, 08-dependent.tex, 09-recursion.tex, 10-contracts.tex, 11-retrieval.tex, 12-ranking.tex, and 15-experts.tex. Same pinned snapshot as [1]. <https://github.com/VladimirReshetnikov/Leant2/tree/784664ff34e819422510a4d470ab07fe48bb736b/docs/proposals/11-further-improvements>.
- [3] Leant2 contributors. *Construction search*, Leant2/Search/Core.lean. Selected static inspection: provider construction, partial instantiation, residual decisions, and transactional alternatives. Same pinned snapshot as [1]. <https://github.com/VladimirReshetnikov/Leant2/blob/784664ff34e819422510a4d470ab07fe48bb736b/Leant2/Search/Core.lean>.
- [4] Leant2 contributors. *Acceptance gate*, Leant2/Accept/Gate.lean. Same pinned snapshot as [1]. <https://github.com/VladimirReshetnikov/Leant2/blob/784664ff34e819422510a4d470ab07fe48bb736b/Leant2/Accept/Gate.lean>.
- [5] Leant2 contributors. *Transactional search state*, Leant2/Native/Transaction.lean. Same pinned snapshot as [1]. <https://github.com/VladimirReshetnikov/Leant2/blob/784664ff34e819422510a4d470ab07fe48bb736b/Leant2/Native/Transaction.lean>.
- [6] Lean contributors. *Induction meta operations*, src/Lean/Meta/Tactic/Induction.lean, tag v4.34.0. Includes getMajorTypeIndices, mkRecursorAppPrefix, InductionSubgoal, and MVarId.induction. <https://github.com/leanprover/lean4/blob/v4.34.0/src/Lean/Meta/Tactic/Induction.lean>.
- [7] Lean contributors. *Library suggestion API*, src/Lean/LibrarySuggestions/Basic.lean, tag v4.34.0. <https://github.com/leanprover/lean4/blob/v4.34.0/src/Lean/LibrarySuggestions/Basic.lean>.
- [8] J. Limperg and A. H. From. Aesop: White-Box Best-First Proof Search for Lean. *CPP*, 2023. doi:10.1145/3573105.3575671. Author copy: <https://people.compute.dtu.dk/ahfrom/aesop-camera-ready.pdf>.
- [9] C. Norman and J. Avigad. Canonical for Automated Theorem Proving in Lean. *ITP*, 2025. Preprint <https://arxiv.org/abs/2504.06239>.
- [10] C. Norman and J. Avigad. Implementing Dependent Type Theory Inhabitation and Unification. Preprint, 2 March 2026, version 1. <https://arxiv.org/html/2603.01463v1>.
- [11] S. Barke, H. Peleg, and N. Polikarpova. Just-in-Time Learning for Bottom-Up Enumerative Synthesis. *OOPSLA*, 2020. doi:10.1145/3428295. <https://arxiv.org/abs/2010.08663>.
- [12] J. Lubin, N. Collins, C. Omar, and R. Chugh. Program Sketching with Live Bidirectional Evaluation. *ICFP*, 2020. doi:10.1145/3408991. Extended paper: <https://arxiv.org/abs/1911.00583>. See the formal-core example restriction and the implementation discussion for the higher-order distinction.
- [13] A. Miltner, A. Trejo Nuñez, A. Brendel, S. Chaudhuri, and I. Dillig. Bottom-up Synthesis of Recursive Functional Programs Using Angelic Execution. *POPL*, 2022. doi:10.1145/3498682. <https://arxiv.org/abs/2107.06253>.
- [14] Z. Guo, M. James, D. Justo, J. Zhou, Z. Wang, R. Jhala, and N. Polikarpova. Program Synthesis by Type-Guided Abstraction Refinement. *POPL*, 2020. doi:10.1145/3371080. Author copy: https://ranjitjhala.github.io/static/hoogle_plus.pdf. See Sections 2.5 and 5.1 for type classes and dictionary passing.
- [15] R. Ji, Y. Zhao, Y. Xiong, D. Wang, L. Zhang, and Z. Hu. Decomposition-Based Synthesis for Applying Divide-and-Conquer-Like Algorithmic Paradigms. *ACM Transactions on Programming Languages and Systems*, 2024. doi:10.1145/3648440. Reviewed version: <https://arxiv.org/html/2202.12193v4>. Artifact: <https://github.com/jiry17/AutoLifter>.

- [16] A. Farzan, D. Lette, and V. Nicolet. Recursion Synthesis with Unrealizability Witnesses. *PLDI*, 2022. doi:10.1145/3519939.3523726. Author copy: <https://www.cs.toronto.edu/~azadeh/resources/papers/pldi22-1.pdf>.
- [17] N. Mulleners, J. Jeuring, and B. Heeren. Example-Based Reasoning about the Realizability of Polymorphic Programs. *ICFP*, 2024. doi:10.1145/3674636. <https://arxiv.org/abs/2406.18304>.
- [18] J. Gibbons, G. Hutton, and T. Altenkirch. When is a Function a Fold or an Unfold? *Electronic Notes in Theoretical Computer Science*, 44(1), 2001. <https://www.cs.ox.ac.uk/publications/publication2368-abstract.html>.
- [19] D. Angluin. Learning Regular Sets from Queries and Counterexamples. *Information and Computation*, 75:87–106, 1987. doi:10.1016/0890-5401(87)90052-6.
- [20] M. Johansson, D. Rosén, N. Smallbone, and K. Claessen. Hipster: Integrating Theory Exploration in a Proof Assistant. *CICM*, 2014. doi:10.1007/978-3-319-08434-3_9. <https://arxiv.org/abs/1405.3426>.
- [21] M. A. Hammer, K. Y. Phang, M. Hicks, and J. S. Foster. Adapton: Composable, Demand-Driven Incremental Computation. *PLDI*, 2014. doi:10.1145/2594291.2594324. Author project and bibliography: <https://matthewhammer.org/adapton/>.
- [22] M. Carneiro. Lean4Lean: Towards a Verified Typechecker for Lean, in Lean. Preprint, 2024. <https://arxiv.org/abs/2403.14064>.
- [23] Lean contributors. *The Lean Language Reference: Recursive Definitions*. Moving reference manual, consulted 21 September 2026. <https://lean-lang.org/doc/reference/latest/Definitions/Recursive-Definitions/>. Source-level implementation recommendations in this report instead use the explicit Lean 4.34 pin.
- [24] Lean contributors. *The Lean Language Reference: Tactic Reference*. Moving reference manual, consulted 21 September 2026. <https://lean-lang.org/doc/reference/latest/Tactic-Proofs/Tactic-Reference/>.
- [25] CoqHammer contributors. *CoqHammer and sauto documentation*. Unfolding controls and automation interfaces; consulted 21 September 2026. <https://coqhammer.github.io/>.
- [26] N. Polikarpova, I. Kuraj, and A. Solar-Lezama. Program Synthesis from Polymorphic Refinement Types. *PLDI*, 2016. doi:10.1145/2908080.2908093.
- [27] M. Balog, A. L. Gaunt, M. Brockschmidt, S. Nowozin, and D. Tarlow. DeepCoder: Learning to Write Programs. *ICLR*, 2017. <https://arxiv.org/abs/1611.01989>.
- [28] K. Ellis et al. DreamCoder: Bootstrapping Inductive Program Synthesis with Wake-Sleep Library Learning. *PLDI*, 2021. <https://arxiv.org/abs/2006.08381>.
- [29] K. Yang et al. LeanDojo: Theorem Proving with Retrieval-Augmented Language Models. *NeurIPS*, 2023. <https://arxiv.org/abs/2306.15626>.
- [30] T. Zhu, J. Clune, J. Avigad, A. Q. Jiang, and S. Welleck. Premise Selection for a Lean Hammer. Preprint initially posted in 2025. <https://arxiv.org/abs/2506.07477>.
- [31] *LeanSearch v2: Global Premise Retrieval for Lean4 Theorem Proving*. Preprint, May 2026; reviewed version 2. <https://arxiv.org/html/2605.13137v2>.